

Университет ИТМО
Факультет программной инженерии и компьютерной техники

Отчёт по групповому проекту
по курсу "Разработка мобильных приложений"

Система отслеживания акций и портфеля

Android-клиент, backend, сервис котировок и observability-контур

Студенты: Беля Алексей Иванович
Хачатрян Геворк Артурович
Шуст Марья Владимировна
Садовой Григорий Владимирович
Состанов Тимур Айратович
Трофимов Владислав Дмитриевич

Преподаватель: Ключев А. О.

Содержание

Введение	3
1 Техническое задание	3
1.1 Введение	3
1.2 Назначение разработки	3
1.3 Требования к программе или программному изделию	4
1.3.1 Функции системы	4
1.3.2 Состав системы	4
1.3.3 Характеристики системы	5
2 Описание архитектуры	5
2.1 Архитектурная идея	5
2.2 Декомпозиция на компоненты	6
2.3 Внутренняя архитектура Kotlin backend	7
2.4 Основные потоки взаимодействия	7
2.4.1 Сценарий аутентификации и торговых операций	7
2.4.2 Сценарий получения котировок	8
2.5 Хранилища и интеграции	8
2.6 Наблюдаемость и OpenTelemetry	8
2.7 Анализ веток и источников проекта	9
2.8 Архитектурные ограничения	9
3 Описание реализации	9
3.1 Android-клиент	9
3.2 Экраны мобильного интерфейса	10
3.3 API gateway	11
3.4 Транзакционный backend	11
3.5 Сервис котировок и инфраструктура	12
3.6 Документация и база знаний	12
4 Описание системы тестирования	13
4.1 Требуемые уровни тестирования	13
4.2 Текущее покрытие тестами	13
4.2.1 Модульное и интеграционное тестирование backend	13
4.2.2 Недостающие области	13
4.3 Предлагаемая стратегия дальнейшего развития	14
4.3.1 Модульное тестирование	14
4.3.2 Интеграционное тестирование	14
4.3.3 Системное тестирование	14
4.4 Нагрузочное тестирование	15
4.5 Тестовое инструментальное обеспечение	16

4.6 Вывод по тестированию	16
Заключение	16

Введение

В рамках дисциплины «Разработка мобильных приложений» был выполнен групповой проект по созданию программной системы для поддержки трейдинга и инвестиций. Разрабатываемая система представляет собой экосистему, включающую Android-приложение и backend-часть, обеспечивающую получение котировок, предоставление информации пользователям и выполнение операций покупки и продажи акций.

Цель работы --- разработать и описать многокомпонентную систему для просмотра котировок, ведения портфеля и выполнения торговых операций с мобильного устройства. Для достижения этой цели были поставлены задачи анализа предметной области, формулирования требований, проектирования архитектуры, реализации клиентской и серверной частей, а также рассмотрения вопросов тестирования и документирования.

1. Техническое задание

1.1. Введение

Разрабатываемая система предназначена для демонстрации архитектурных и прикладных аспектов мобильной разработки: пользовательский интерфейс Android-приложения, взаимодействие с backend API, работа с котировками, хранение транзакционных и аналитических данных, а также сбор telemetry-данных для диагностики распределённой системы.

Проект рассматривается как учебная торгово-информационная платформа. Пользователь системы должен иметь возможность зарегистрироваться, войти в приложение, получить актуальную котировку по тикеру, выполнить покупку или продажу актива и увидеть агрегированную статистику своего портфеля.

1.2. Назначение разработки

Назначение разработки заключается в создании многомодульного мобильного программного комплекса, который:

- предоставляет Android-клиент для взаимодействия с пользователем;
- предоставляет backend API для аутентификации и операций с портфелем;
- получает и публикует рыночные котировки;
- демонстрирует работу распределённой архитектуры с observability-подсистемой;
- обеспечивает базу для дальнейшего расширения системы тестирования и документации.

1.3. Требования к программе или программному изделию

1.3.1. Функции системы

Система должна обеспечивать следующие функции:

1. Регистрация пользователя по логину и паролю.
2. Аутентификация пользователя и выдача access token.
3. Пополнение денежного баланса пользовательского портфеля.
4. Просмотр актуальной котировки по тикеру.
5. Просмотр информации о владении выбранной акцией в портфеле.
6. Выполнение операций покупки и продажи акций.
7. Получение агрегированной статистики портфеля.
8. Ведение истории котировок и публикация событий в инфраструктурные каналы.
9. Сбор трассировок и технических данных для анализа работы сервисов.

1.3.2. Состав системы

В состав системы входят:

- Android-клиент android-app/;
- публичный API gateway mobile-api/;
- транзакционный сервис портфеля mobile-backend/;
- сервис котировок quotes-service/;
- Linux C-драйвер driver/ как низкоуровневый источник котировок;
- PostgreSQL для транзакционных данных;
- Redis для событий и latest-cache;
- ClickHouse для истории котировок;
- OpenTelemetry Collector, Tempo, Prometheus, Loki и Grafana для наблюдаемости;
- документация проекта и база знаний в Obsidian.

1.3.3. Характеристики системы

К системе предъявляются следующие характеристики:

- модульность и разделение ответственности между клиентом, gateway, backend и сервисом котировок;
- возможность локального запуска в Docker Compose;
- расширяемость по API, хранилищам и observability;
- поддержка трассировки запросов через OpenTelemetry;
- наличие автоматизированных тестов хотя бы для части backend-компонентов;
- документированность архитектуры, реализации и тестирования;
- возможность дальнейшей интеграции Android-клиента с реальным backend API.

2. Описание архитектуры

2.1. Архитектурная идея

Ключевая идея проекта состоит в разделении системы на несколько относительно независимых подсистем. Такое разделение позволяет не смешивать клиентский UI, бизнес-логику портфеля, рыночные данные, техническую инфраструктуру и observability. Для учебного проекта это особенно важно, так как даёт возможность параллельной работы нескольких участников команды и упрощает понимание внутренних границ системы.

В актуальном состоянии архитектура имеет сервисный характер:

- Android-клиент отвечает за пользовательский интерфейс;
- mobile-api является внешней точкой входа для мобильных приложений;
- mobile-backend реализует бизнес-логику аутентификации, портфеля и торговых операций;
- quotes-service предоставляет актуальные котировки и историю;
- инфраструктурный слой хранит данные и собирает телеметрию.

На рисунке показана целевая backend-схема проекта, зафиксированная в документации и текущей структуре репозитория. Пользовательский сценарий начинается в Android-клиенте, проходит через внешний gateway mobile-api и попадает во внутренний транзакционный сервис mobile-backend. Отдельно выделен контур рыночных данных: quotes-service получает котировки от Linux C-драйвера, публикует актуальные значения в Redis и сохраняет историю в ClickHouse. Также схема отражает observability-подсистему, где backend-сервисы экспортируют telemetry-данные в OpenTelemetry Collector и далее в стек мониторинга.

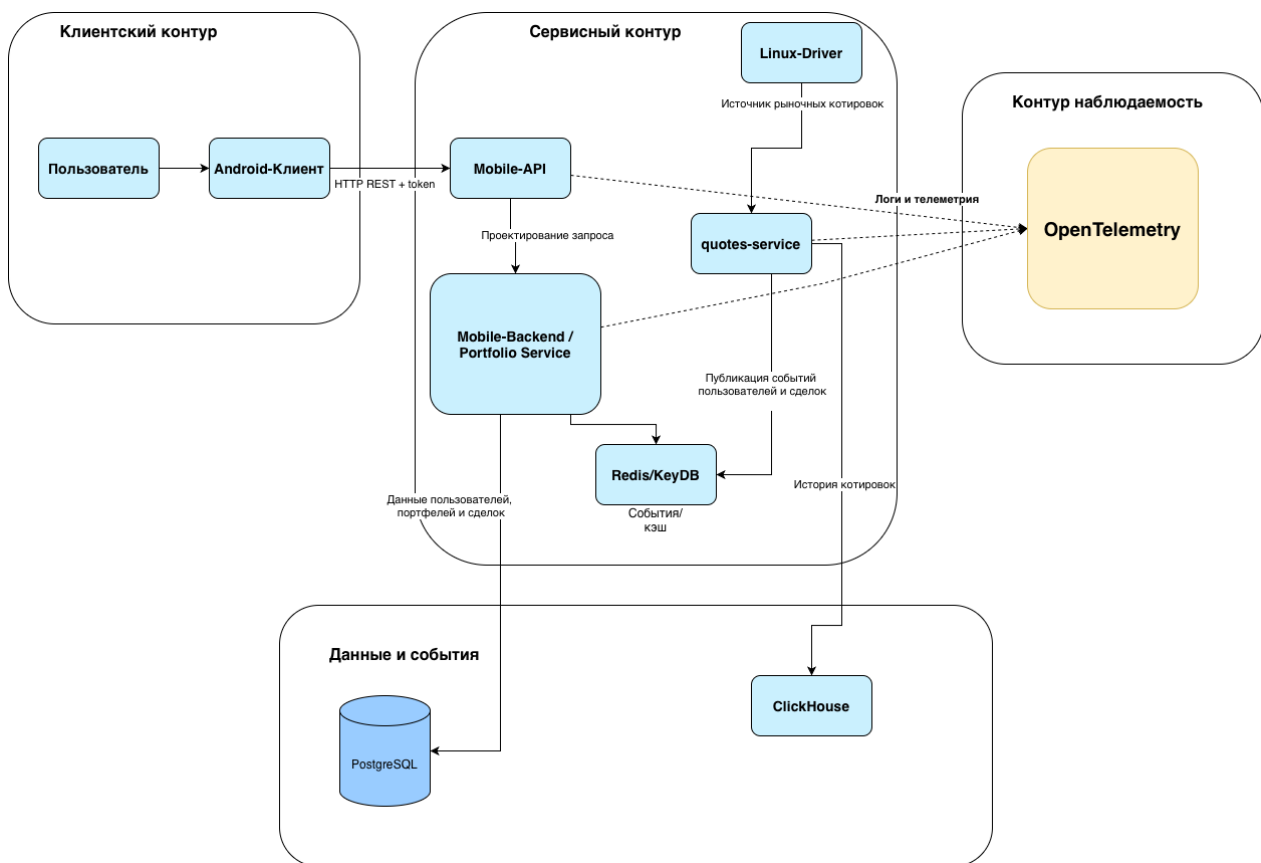


Рис. 1: Архитектура клиент-серверной системы проекта

2.2. Декомпозиция на компоненты

Компонент	Архитектурная роль
android-app	Android-прототип с экранами логина, профиля, статистики, списка акций и карточки акции. Пока интегрирован с backend лишь концептуально.
mobile-api	Ktor gateway, который принимает клиентские HTTP-запросы, выполняет общие middleware-функции и проксирует запросы во внутренние сервисы.
mobile-backend	Транзакционный сервис портфеля. Управляет пользователями, JWT, операциями покупки и продажи, статистикой и взаимодействием с PostgreSQL.
quotes-service	Go-сервис чтения котировок, поддерживающий mock-источник и Linux-драйвер, а также публикацию данных в Redis и ClickHouse.
driver	Симуляция устройства /dev/quotes на стороне Linux-хоста. Нужен как низкоуровневая часть учебной задачи по источнику рыночных данных.
postgres	Основное хранилище пользователей, портфелей, lot-структур и истории сделок.

Компонент	Архитектурная роль
redis	Канал Streams для событий и быстрый storage latest-cache по котировкам.
clickhouse	Хранилище истории котировок, удобное для аналитических и временных запросов.
otel-collector и related stack	Приём и маршрутизация telemetry-сигналов от сервисов, интеграция с Tempo, Prometheus, Loki и Grafana.

2.3. Внутренняя архитектура Kotlin backend

Сервис mobile-backend построен по модели слоистой архитектуры, близкой к Clean Architecture [2]. Внутренние пакеты разделены на:

- presentation --- HTTP-маршруты, DTO, плагины Ktor, обработка ошибок;
- application --- use case и порты;
- domain --- предметные сущности и value objects;
- infrastructure --- БД, безопасность, Redis, OpenTelemetry, HTTP-клиенты и конфигурация.

Такое разделение делает use case слабее связанными с Ktor, JDBC и конкретными внешними библиотеками. В результате тестирование логики регистрации, торговли и статистики возможно без полного запуска серверного окружения.

2.4. Основные потоки взаимодействия

Типовой пользовательский сценарий на уровне HTTP-взаимодействий проходит через несколько компонентов. Для входа пользователя mobile-api выступает только как внешний шлюз, а проверка учётных данных и выдача JWT выполняется во внутреннем сервисе mobile-backend. Для защищённых операций backend проверяет JWT, обращается к PostgreSQL за состоянием портфеля и, при необходимости, запрашивает актуальную цену у quotes-service. После выполнения покупки backend публикует событие trade.executed в Redis, а затем возвращает ответ обратно через gateway в мобильное приложение.

2.4.1. Сценарий аутентификации и торговых операций

1. Клиент отправляет запрос в mobile-api.
2. mobile-api создаёт trace/span, фиксирует request context и пересылает запрос в mobile-backend.
3. mobile-backend выполняет use case, при необходимости обращается к PostgreSQL и публикует событие в Redis Streams.

4. Ответ возвращается назад через mobile-api.

2.4.2. Сценарий получения котировок

1. quotes-service регулярно считывает снапшоты котировок из mock-файла или из /dev/quotes.
2. Сервис обновляет in-memory state, latest-cache в Redis и историю в ClickHouse.
3. mobile-backend обращается к quotes-service за актуальной ценой.
4. mobile-api возвращает котировку мобильному клиенту.

2.5. Хранилища и интеграции

Хранилища выбраны не случайно, а по ролям:

- PostgreSQL хранит транзакционные данные, где важны консистентность и работа с отношениями;
- Redis используется для Streams и быстрого latest-cache;
- ClickHouse хранит историю котировок как аналитический временной ряд.

Такое разделение упрощает дальнейший рост проекта: транзакционные и аналитические нагрузки не смешиваются, а события можно использовать для новых фоновых сервисов.

2.6. Наблюдаемость и OpenTelemetry

В проекте уже реализован базовый контур наблюдаемости:

- mobile-api создаёт серверные span на HTTP-запросы;
- mobile-backend использует OpenTelemetryTelemetryRecorder для внутренних span;
- quotes-service подключает OTLP exporter для Go;
- otel-collector принимает traces, metrics и logs и передаёт traces в Tempo [6].

Архитектурно это важно по двум причинам. Во-первых, observability встроена в систему не как внешняя надстройка, а как часть её runtime-модели. Во-вторых, наличие gateway и нескольких backend-компонентов делает распределённую трассировку действительно полезной.

2.7. Анализ веток и источников проекта

При подготовке архитектурного описания важно учитывать историю репозитория. В проекте присутствуют ветки `origin/feature/market-data` и `origin/feature/mobile-backend-db`, однако обе они уже не содержат уникальных коммитов относительно актуального `origin/main`. Следовательно, они представляют ранние этапы развития backend, но не определяют текущее состояние системы.

Android-клиент, наоборот, не находился в общем Git-источнике проекта и был импортирован отдельно из внешнего репозитория команды. Поэтому архитектурно проект сейчас состоит из двух линий происхождения: основной backend-контур из общего репозитория и отдельно созданный клиентский Android-прототип. Это необходимо честно фиксировать в отчёте, чтобы не возникало ложного впечатления, будто весь проект развивался в одном и том же дереве исходников с самого начала.

2.8. Архитектурные ограничения

Несмотря на сильную серверную часть, есть и важные ограничения:

- Android-клиент пока не работает с реальным API проекта;
- в карточке акции покупка и продажа ещё не привязаны к backend;
- часть UI-статистики и графиков работает на моках;
- у mobile-api пока нет собственного тестового набора.

Эти ограничения не отменяют архитектуру, но показывают, где проходит граница между уже реализованным контуром и следующим этапом интеграции.

3. Описание реализации

3.1. Android-клиент

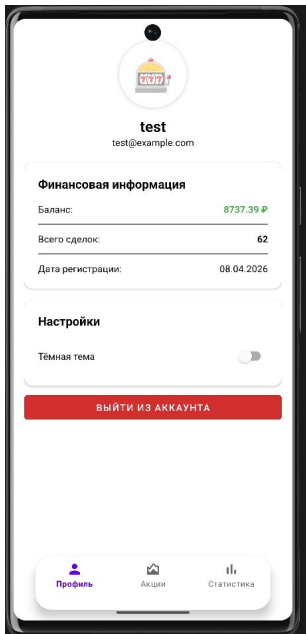
Импортированный Android-клиент реализован на Kotlin с использованием классического стека Android Views, XML layouts, Activity, Fragment, RecyclerView и BottomNavigationView [4]. Основные точки входа:

- MainActivity --- экран входа;
- SecondActivity --- контейнер нижней навигации;
- ProfileFragment, StatisticsFragment, StockListFragment, StockChartFragment --- основные пользовательские экраны.

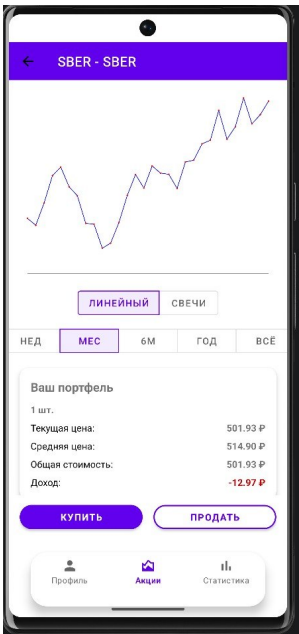
Текущая реализация клиента носит прототипный характер. Вход реализован локальной проверкой непустых полей, котировки берутся напрямую из MOEX ISS через `HttpURLConnection`, а в части экранов используются локально сгенерированные значения. Тем не менее экранная структура уже хорошо подготовлена для подключения реального backend API.

3.2. Экраны мобильного интерфейса

Для отчёта были подготовлены четыре ключевых скриншота Android-клиента. Они показывают текущий уровень проработки интерфейса и помогают соотнести описание реализации с реальными пользовательскими экранами.

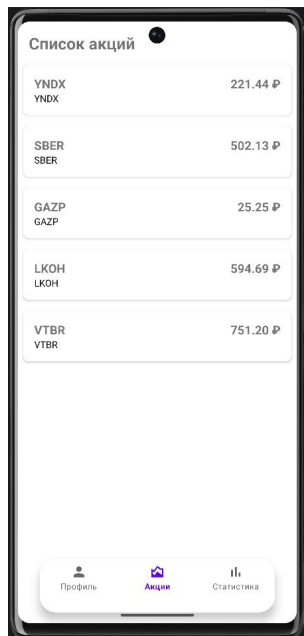


Экран профиля пользователя

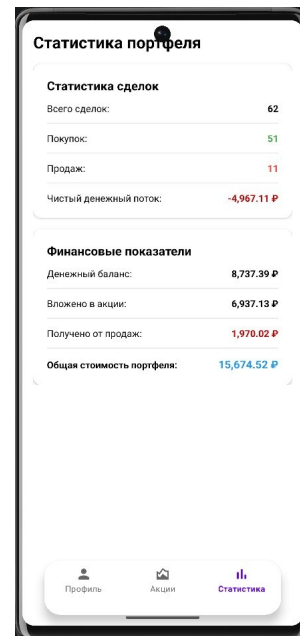


Экран карточки акции Сбербанка

Рис. 2: Экраны профиля пользователя и карточки акции



Экран списка акций



Экран статистики портфеля

Рис. 3: Экраны списка акций и статистики портфеля

3.3. API gateway

Модуль mobile-api реализован на Ktor [3]. Его ApplicationModule выполняет:

- загрузку конфигурации;
- создание OpenTelemetry handle;
- создание HTTP-клиента для upstream;
- подключение Ktor plugins: call id, logging, content negotiation, tracing, status pages, routing.

Маршруты gateway повторяют клиентский API-контракт и проксируют вызовы к mobile-backend. Такой слой полезен не только как reverse проху, но и как архитектурный буфер между мобильным приложением и внутренними сервисами.

3.4. Транзакционный backend

Модуль mobile-backend является самым зрелым функциональным слоем проекта. В bootstrap создаются:

- use case аутентификации;
- use case пополнения денежного баланса;
- use case торговли;
- use case чтения деталей владения;

- use case получения статистики;
- репозитории Exposed;
- publisher событий в Redis Streams;
- компонент для записи telemetry-событий;
- JWT и password hashing инфраструктура.

Из реализованных HTTP-операций в backend доступны регистрация, вход, пополнение денежного баланса, получение котировки, получение информации по акции, покупка, продажа и статистика портфеля. После обновлений апреля 2026 года backend также проверяет достаточность средств перед покупкой и возвращает корректный пустой ответ по holdings, если запрошенный символ отсутствует в существующем портфеле. На текущем этапе это ядро серверной части всей системы.

3.5. Сервис котировок и инфраструктура

quotes-service реализован на Go и запускает HTTP-сервер с маршрутом /health, /metrics, /quotes, /quotes/{ticker} и /quotes/{ticker}/history. Сервис умеет:

- читать котировки из mock-файла или из Linux-источника;
- держать актуальное состояние в памяти;
- публиковать latest-state и события в Redis;
- сохранять историю в ClickHouse;
- обрабатывать HTTP-обработчики через OpenTelemetry middleware.

Верхний уровень инфраструктуры описан в docker-compose.yml. Он поднимает caddy, postgres, redis, clickhouse, quotes-service, portfolio-service, mobile-api, otel-collector, loki, tempo, prometheus и grafana. Таким образом, проект уже имеет полноценный локальный стенд для разработки и демонстрации.

3.6. Документация и база знаний

В рамках подготовки сдачи были заново сформированы:

- проектная документация в каталоге docs/;
- база знаний Obsidian в каталоге knowledge-base/obsidian/;
- исходный LaTeX-текст отчёта и PDF-сборка.

Такой набор материалов соответствует требованиям курса: знания фиксируются не только в коде, но и в поясняющих артефактах, которые можно использовать при защите и передаче проекта внутри команды.

4. Описание системы тестирования

4.1. Требуемые уровни тестирования

Согласно лекционным материалам, в отчёте должны быть отражены три уровня тестирования: модульное, интеграционное и системное [1]. В текущем проекте это особенно важно, так как система распределена по нескольким технологиям и языкам.

4.2. Текущее покрытие тестами

4.2.1. Модульное и интеграционное тестирование backend

Наиболее развито тестирование в mobile-backend. В репозитории присутствуют тесты для:

- регистрации пользователя;
- покупки акций и проверок денежного баланса;
- получения рыночной котировки;
- чтения деталей владения акцией;
- расчёта статистики портфеля;
- логики продажи акций;
- репозитория пользователей;
- HTTP-маршрутов аутентификации;
- HTTP-маршрутов рыночных данных.

Для quotes-service присутствуют тесты парсинга, store и HTTP-обработчиков. Следовательно, серверная часть уже имеет реальную основу автоматизированной проверки, а не только декларативное описание.

4.2.2. Недостающие области

У mobile-api подключены зависимости для тестирования, но собственного каталога src/test пока нет. У android-app присутствуют шаблонные зависимости для unit и instrumentation testing, однако содержательных тестов пока не реализовано.

4.3. Предлагаемая стратегия дальнейшего развития

4.3.1. Модульное тестирование

Должно покрывать:

- use case backend;
- парсинг и преобразование котировок;
- DTO и data layer Android-клиента после интеграции с API;
- вспомогательные функции observability и request context.

4.3.2. Интеграционное тестирование

Должно покрывать:

- mobile-api + mock upstream;
- mobile-backend + тестовая БД;
- quotes-service + тестовые Redis/ClickHouse doubles;
- Android data layer + mock web server.

4.3.3. Системное тестирование

Минимальный end-to-end сценарий должен включать:

1. регистрацию нового пользователя;
2. вход и получение JWT;
3. пополнение денежного баланса;
4. запрос актуальной котировки;
5. выполнение операции покупки;
6. чтение статистики портфеля;
7. проверку появления trace в observability stack.

4.4. Нагрузочное тестирование

В качестве отдельного примера системной проверки была выполнена серия нагрузочных прогонов публичного endpoint получения котировки. Использовался сценарий на 10000 запросов с уровнем параллелизма 100 к endpoint <https://song-analysis.app/market/quotes/AAPL>. Всего было выполнено 10 запусков, то есть суммарно отправлено 100000 запросов.

Один из прогонов в условиях мобильного интернета с включённым VPN показал следующие результаты:

- общее число запросов --- 10000;
- успешных запросов, включая обработанные приложением ошибки, --- 9996;
- неуспешных сетевых или необработанных запросов --- 4;
- общее время выполнения --- 137273 мс;
- производительность --- 72.85 запросов в секунду.

Прогон без VPN в тех же сетевых условиях дал следующий результат:

- общее число запросов --- 10000;
- успешных запросов, включая обработанные приложением ошибки, --- 9852;
- неуспешных сетевых или необработанных запросов --- 148;
- общее время выполнения --- 108647 мс;
- производительность --- 92.04 запросов в секунду.

По серии из 10 запусков средние показатели составили:

- суммарное число запросов --- 100000;
- суммарное число успешных запросов --- около 99240;
- суммарное число неуспешных сетевых или необработанных запросов --- около 760;
- среднее время одного прогона --- около 123000 мс;
- средняя производительность --- около 82.45 запросов в секунду.

Эти результаты нельзя считать исчерпывающим профилированием всей платформы, однако серия прогонов подтверждает, что внешний API-слой способен выдерживать интенсивную однотипную нагрузку порядка 10000 запросов за запуск даже в нестабильных сетевых условиях мобильного интернета.

4.5. Тестовое инструментальное обеспечение

Для системных и интеграционных сценариев полезно разработать небольшое инструментальное ПО на Kotlin, Go или Python, как это и предлагается в материалах курса [1]. Практически это может быть:

- CLI-клиент или набор сценариев для вызова API;
- нагрузочный генератор пользовательских сессий;
- набор smoke-сценариев для проверки Docker-стенда;
- утилита проверки наличия trace/span после сценария сделки.

4.6. Вывод по тестированию

Система тестирования уже имеет работающую основу в backend- и Go-частях, но в целом ещё не завершена. Для полного соответствия архитектуре проекта необходимо выровнять тестовую зрелость mobile-api и Android-клиента и добавить сквозные системные сценарии.

Заключение

В результате выполнения курсовой работы была разработана многокомпонентная программная система для поддержки трейдинга и инвестиций, объединяющая мобильное приложение и серверную часть. Разработанное решение обеспечивает получение рыночных котировок, ведение инвестиционного портфеля и выполнение базовых операций покупки и продажи акций, что позволило достигнуть поставленной цели работы.

В ходе проекта были сформулированы требования к системе, определён состав её компонентов, спроектирована архитектура, реализованы ключевые элементы клиентской и серверной частей, а также рассмотрены вопросы тестирования и документирования. Практическая значимость работы заключается в том, что созданное решение представляет собой не отдельное учебное приложение, а систему взаимодействующих компонентов, близкую по своей структуре к реальным мобильным финансовым сервисам.

Список литературы

- [1] Ключев А. О. Разработка мобильных приложений. Лекция 1: введение и практическое задание. Учебные материалы курса, 2026.
- [2] Martin R. C. Clean Architecture: A Craftsman's Guide to Software Structure and Design. Boston: Prentice Hall, 2017.
- [3] JetBrains. Ktor Documentation [Электронный ресурс]. URL: <https://ktor.io/docs/server-create-and-configure.html> (дата обращения: 01.04.2026).

- [4] Android Developers. Activities and Fragments [Электронный ресурс]. URL: <https://developer.android.com/guide/fragments> (дата обращения: 01.04.2026).
- [5] JetBrains. Exposed Documentation [Электронный ресурс]. URL: <https://www.jetbrains.com/help/exposed/about.html> (дата обращения: 01.04.2026).
- [6] OpenTelemetry. Collector and Tracing Documentation [Электронный ресурс]. URL: <https://opentelemetry.io/docs/collector/> (дата обращения: 01.04.2026).
- [7] Redis. Streams [Электронный ресурс]. URL: <https://redis.io/docs/latest/develop/data-types/streams/> (дата обращения: 01.04.2026).
- [8] ClickHouse. HTTP Interface Documentation [Электронный ресурс]. URL: <https://clickhouse.com/docs/en/interfaces/http> (дата обращения: 01.04.2026).
- [9] Документация проекта stock-tracker-app: README.md, docs/system-architecture.md, docs/backend-implementation.md, docs/endpoints.md. Репозиторий команды, состояние на 08.04.2026.